AutoGrader: A Rule-Driven Multi-Agent Evaluator for C Programs

Local Ollama Intelligence with React, FastAPI and Sandboxed Execution

TEAM EVEREST

Kanishk Mishra
2611CS20

Roshan Jha
2611CS23

Mantosh Ravi
2611CS09

Shivanshu Raj
2611CS26

Abstract

This paper presents AutoGrader, a local, rule-driven system for evaluating student C programs using a combination of agentic AI and traditional software engineering. An instructor supplies a source file and an external JSON rubric. A Python coordinator performs compilation and smoke checks, delegates nine grading metrics to independent analyzers, and combines their scores using validated weights. Deterministic tests, a trusted edge-case oracle, compiler diagnostics and sanitizer evidence complement four local Ollama specialists for structure, modularity, complexity and readability. A React frontend communicates with a FastAPI backend to expose editable rules, evaluation status, feedback, report history and JSON/text downloads. The implementation uses direct file handling rather than retrieval-augmented generation. A content-based verdict cache supports reproducible replay under a fixed configuration. Verification on the creation host recorded 34 passing unit/API tests and nine skipped target-environment tests. Consequently, this paper reports implemented functionality and software-level verification, while real Docker/Ollama grading and calibration remain Ubuntu acceptance tasks.

Keywords: Agentic AI; automated grading; C programming; Ollama; multi-agent evaluation; React; FastAPI; sandboxing; reproducibility.

1. Introduction and project scope

Manual code assessment becomes difficult when an instructor must judge functional correctness, code quality, efficiency and unsafe behavior across many submissions. The assignment [1] asks for unattended evaluation governed by external rules, independent analyzers, bounded execution and specific failure examples. AutoGrader addresses this need through an evidence-first workflow in which model judgments supplement, rather than replace, executable checks.

The current scope is one UTF-8 C file, a local instructor interface and Ubuntu-based evaluation. The principal contribution is the integration of configurable grading, local semantic review, isolated execution and inspectable reports in a modular codebase. This is an implementation/progress paper, not a claim of measured model accuracy or a production-grade public examination service.

2. System architecture and evaluation flow

The system separates presentation, transport, orchestration and analysis. React handles interaction; FastAPI validates requests and exposes asynchronous job status; the grading service loads files; the coordinator controls all execution and aggregation. Student source is evidence, never an instruction source for tools or the evaluator.

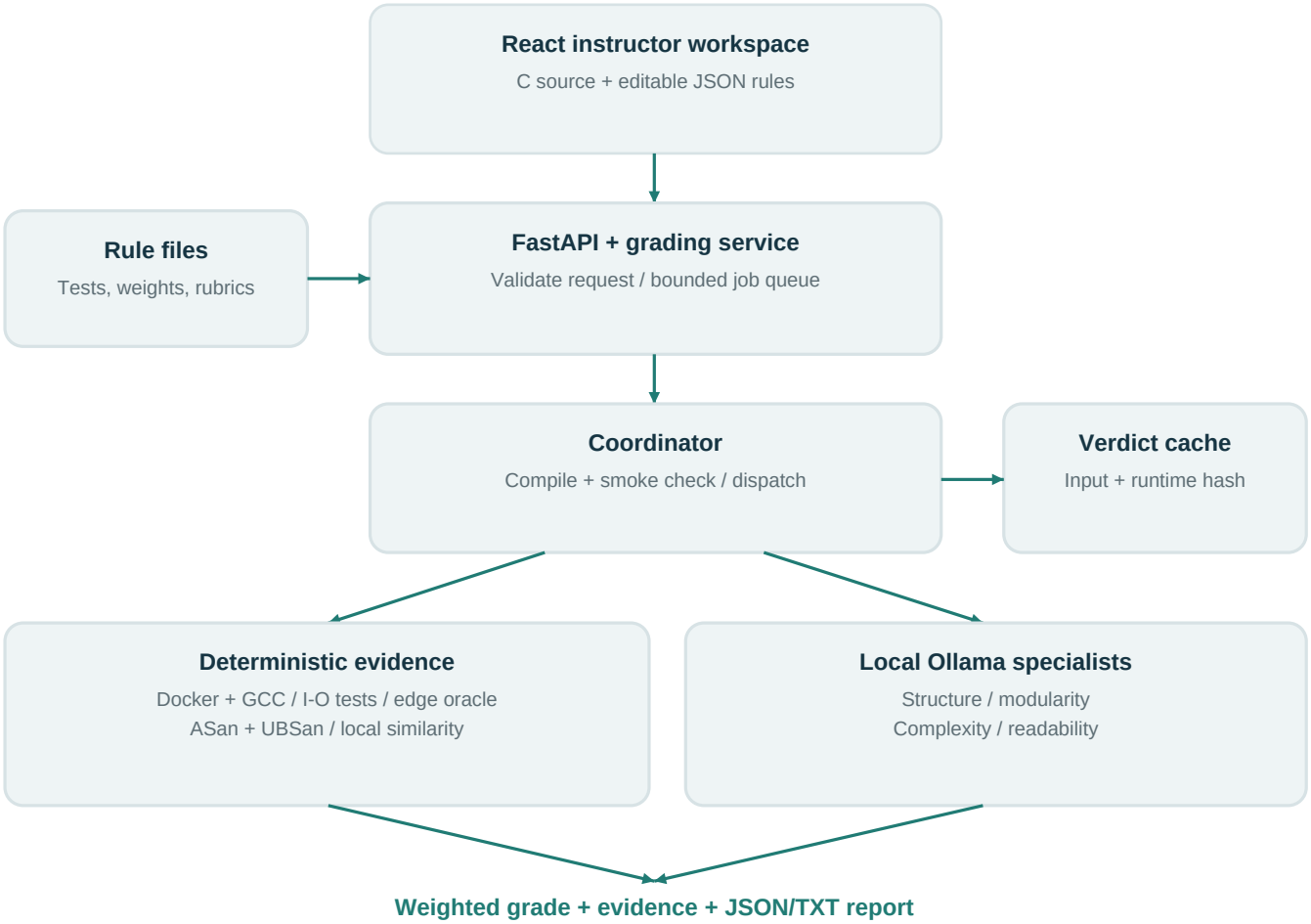

Figure 1. Implemented architecture. Deterministic checks and four independent Ollama conversations feed one weighted report. External rules configure the workflow; a fingerprinted cache replays completed verdicts.

2.1 Coordinator procedure

Input validation: validate source size, metric names, weights, score bounds and JSON structure. Resolve the configured model digest and Docker image identity before consulting the cache. **First pass:** compile with GCC, run the first test as a smoke check and record advisory source patterns. **Dispatch:** invoke the selected analyzers, preserving their evidence and feedback. **Collation:** compute weighted contributions and save machine-readable JSON and student-readable text.

2.2 Failure handling and bounded coordination

A compilation or smoke failure skips execution-based metrics with zero credit at their original weights while source reviewers may still run. Infrastructure or model errors make the overall grade unavailable rather than silently awarding or deducting marks. The API permits one active evaluation and one waiting request. Specialist dispatch is sequential to avoid local-model memory contention and interference with timed execution.

3. Modules, analyzer types and responsibilities

The implementation contains nine metric analyzers. Four are local LLM review agents; five use deterministic tool, oracle or corpus evidence. Every analyzer returns a common result contract: score, status, feedback and raw evidence. Table 1 maps the assignment metrics to their implementation.

Metric / weight	Analyzer type and implementation	Evidence and responsibility
Correctness / 20%	Deterministic runner `correctness.py`	Compare actual output with instructor test outputs; preserve failing inputs and expected answers.
Syntax / 10%	Compiler specialist `syntax.py`	Strict C11 compilation with warning flags and `-Werror`.
Structure / 10%	Ollama rubric agent `structure.py`	Estimate function length, decision complexity and nesting using source-line findings.
Modularity / 10%	Ollama rubric agent `modularity.py`	Assess helper functions, responsibility separation and meaningful decomposition.
Logical / 10%	Trusted oracle runner `edges.py`	Generate boundary and seeded random cases; compute expected sums using Python integers.
Complexity / 10%	Ollama rubric agent `complexity.py`	Estimate asymptotic behavior against a required bound; do not claim a measured proof.
Memory safety / 10%	Instrumented runner `memory.py`	Compile and run with ASan/UBSan; report diagnostics for exercised paths.
Readability / 10%	Ollama rubric agent `readability.py`	Apply explicit naming, layout, comments and clarity anchors.
Originality / 10%	Local similarity check `originality.py`	Compare token sequences with saved reference programs; flag matches for review.

Table 1. All analyzer files are under grader/analyzers/. A generic llm_rubric registry entry also supports additional subjective metrics through JSON configuration.

3.1 Supporting software modules

Frontend: `web/src/main.jsx` and `styles.css`. **API:** `api.py`. **Service:** `backend.py`. **Entry point:** `app.py`. Shared contracts, file operations, model transport, sandbox invocation, locking and reporting are separated inside the `grader/` package. The optional older Gradio frontend is retained separately; React is the default.

The design uses files directly because source, rules and the bounded comparison corpus fit the evaluation workflow without an embedding index. A new tool-backed capability requires an analyzer class and registry entry; a JSON rule cannot execute arbitrary Python or shell commands.

4. Grading methodology and agent configuration

4.1 Weighted scoring

Each metric score s_i lies in [0, 1], and its weight w_i lies in [0, 1]. The rule validator requires the weights to sum to 1. The system does not silently normalize invalid weights.

Overall grade = 100 x sum(w_i x s_i), with sum(w_i) = 1.

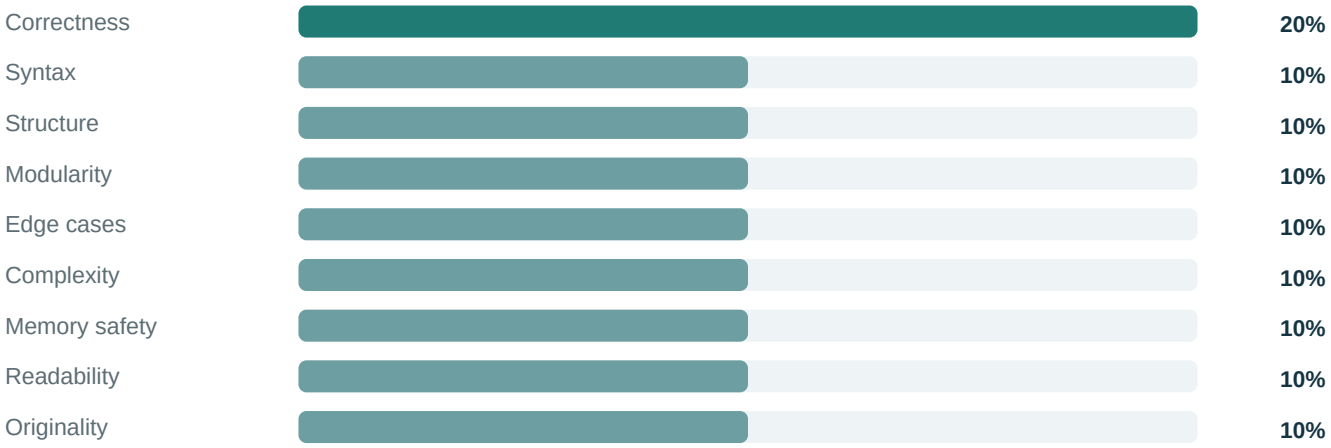

Figure 2. Configured metric weights from rules/add.json. Correctness contributes up to 20 marks; each remaining metric contributes up to 10 marks. These are rubric weights, not observed student results.

4.2 Local evaluator settings

Setting	Configured value / purpose
Model	qwen2.5-coder:7b through the local Ollama HTTP endpoint.
Temperature / seed	0 / 42; reduces sampling variability but does not prove fresh-run determinism.
Context / response limit	16,384 context setting and 2,048 generated-token setting; oversized review input is rejected.
Response contract	Schema-constrained JSON validated with Pydantic; score range and cited source lines are checked.
Prompt boundaries	Role + specification + rubric + numbered source + compiler evidence. Student comments are untrusted data.

4.3 Reproducibility and originality policy

Completed verdicts are cached using source, effective rules/tests, corpus, prompt, grader implementation and runtime identities. This supports repeatable replay for an unchanged evaluation key. It does not guarantee identical new inference across hardware or model versions. Originality defaults to an advisory flag with no automatic misconduct penalty. An empty corpus provides no authorship evidence; similarity does not establish copying or LLM generation.

The assignment illustration contains inconsistent arithmetic: its listed contributions total 79 rather than 89, and 8/10 passing tests imply a score of 0.8. The implementation computes actual fractions and weighted totals instead of reproducing those illustrative values [1].

5. Code implementation and JSON interface

Frontend and backend exchange JSON. The API returns HTTP 202 with a job identifier; the browser polls real queued/running/completed/error states. A completed job carries the assignment-style report and its text representation. Transport completion does not guarantee a valid grade: an incomplete evaluation has `grade: null`.

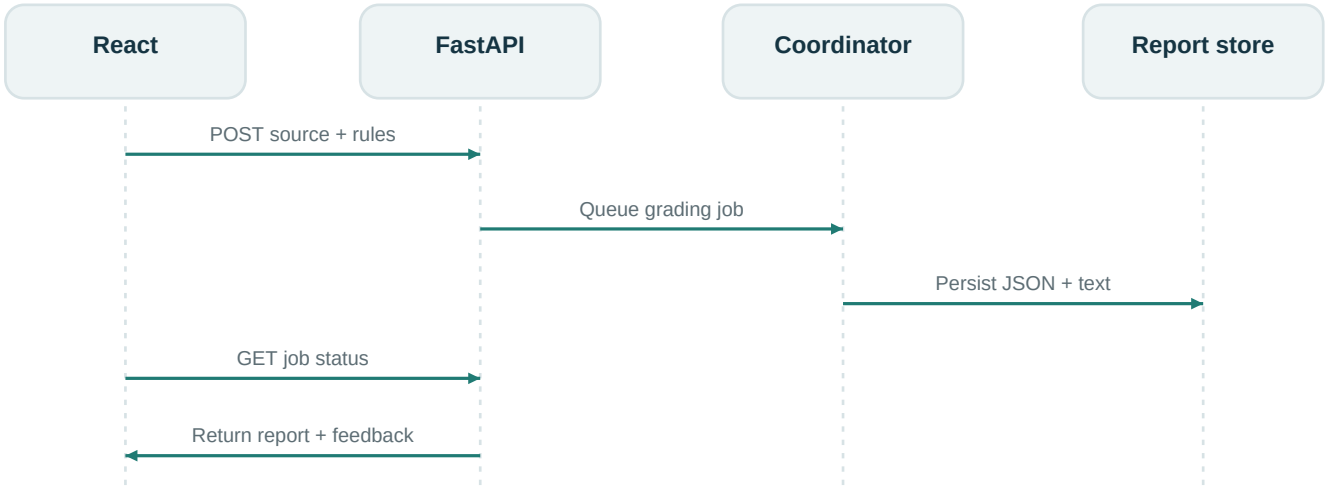

Figure 3. Request/report sequence. The backend preserves the assignment report schema inside the API job response.

5.1 Aggregation logic from the coordinator

```
# grader/coordinator.py: infrastructure errors prevent an official grade.
def aggregate(metrics):
    incomplete = any(m["status"] == "error" for m in metrics)
    points = sum(m["contribution"] for m in metrics)
    return None if incomplete else round(points, 4), round(points, 4)
```

5.2 Example correction: arithmetic width

```
/* examples/add_correct.c: wider arithmetic for int32 inputs. */
static long long add(long long left, long long right) {
    return left + right;
}
```

For the configured problem, inputs are signed 32-bit integers but output is their mathematical sum. Thus `2147483647 1` requires `2147483648`. Signed C overflow is not a guaranteed wraparound result. The provided corrected sample reads and computes with `long long`; this is valid within the stated input domain.

5.3 Report contract and code organization

The report contains `submission`, `rules_hash`, `grade`, `first_pass`, `metrics`, `flags`, `usefulness_limitations` and `tool_versions`. Each metric records score, weight, contribution, evidence and feedback. Additional audit fields identify the verdict, runtime, rules snapshot and cache state. Reports are persisted under a separate evaluation directory, keeping familiar submission-based JSON/TXT filenames.

6. Verification, progress and experimental limits

Software checks were executed on the Windows creation host. Real grading targets Ubuntu, and Docker/Ollama were unavailable on that host. The evidence below therefore distinguishes successful software-level checks from unexecuted runtime/model tests [2].

Figure 4. Test inventory: 43 discovered tests, 34 passed and nine skipped. The skipped groups are eight Docker integration tests and one Docker/Ollama end-to-end test. This chart is not model accuracy or student pass-rate data.

Verified area	Observed evidence
Rules and scoring	Weight totals, duplicate metrics, score bounds, temperature constraints, unknown analyzers and weighted aggregation.
Coordinator behavior	Compilation/smoke dispatch, timeout flags, concrete failure evidence, incomplete reports and cache invalidation/replay.
API and frontend contract	Job lifecycle, rules loading, JSON report fields, size/path/origin checks, visible service errors and edited-rule forwarding.
Build and browser	React production build succeeded; FastAPI served HTTP 200; source/rubric display, rule validation and navigation were checked.

6.1 Target-host checks still required

Ubuntu acceptance must verify compilation failure, correct overflow-boundary output, timeout handling, crashes, output flooding, memory limits, UBSan overflow, supervisor isolation and actual Ollama evaluation with replay. Unit/API tests use explicit test doubles where necessary; they do not demonstrate model correctness or real sandbox behavior. Genuine sample reports must be generated with `run_samples.py` after these prerequisites are ready.

6.2 Representative oracle cases

Input	Expected output	Purpose
7 5	12	Ordinary addition
-7 -5	-12	Negative operands
2147483647 1	2147483648	Upper int32 overflow boundary
-2147483648 -1	-2147483649	Lower int32 overflow boundary

Table 2. Expected mathematical outputs defined by the example problem contract. These are oracle cases, not claimed execution results.

7. Team organization and module work plan

Team name: TEAM EVEREST. The following ownership allocation is proposed for collaboration and review. The names and roll numbers are supplied by the team; the assignment of responsibilities below is a planning recommendation, not a verified record of individual past contributions.

Team member	Proposed module ownership	Review / deliverable
Kanishk Mishra **2611CS20**	Rules, data contracts, coordinator, scoring and verdict cache.	Review rubric validity and aggregation; maintain architecture and reproducibility notes.
Roshan Jha **2611CS23**	Docker worker, compilation, I/O correctness, edge cases and memory-safety checks.	Run Ubuntu timeout/crash/sanitizer tests; document concrete counterexamples.
Mantosh Ravi **2611CS09**	Ollama client, evaluator prompts, structure, modularity, complexity and readability specialists.	Calibrate rubric deductions against instructor-scored examples; review uncertain model findings.
Shivanshu Raj **2611CS26**	React frontend, FastAPI integration, JSON/TXT reporting and UI/API tests.	Verify upload-to-report workflow, report history, documentation and demonstration materials.

7.1 Suggested four-week schedule

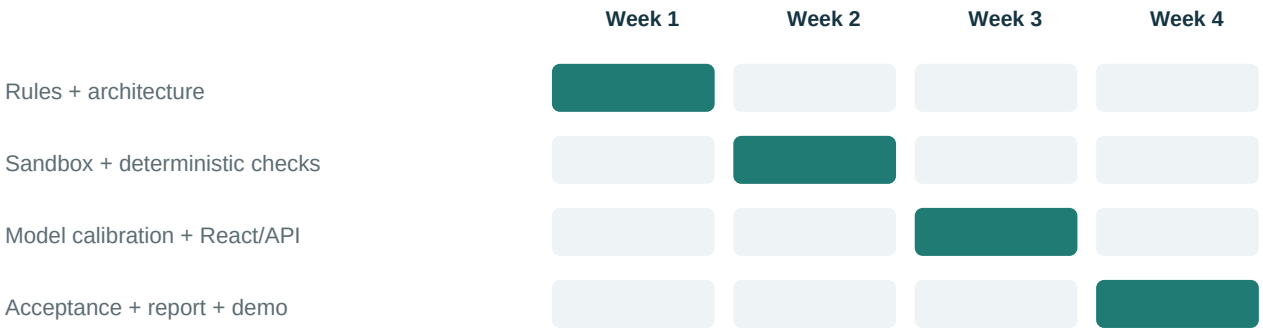

Figure 5. Proposed work schedule aligned with the assignment's four-week project format. It is a planning chart, not an actual timesheet or a measured completion percentage.

7.2 Current status and remaining delivery work

Implemented: modular Python analyzers, rules and prompts, Docker worker, React/FastAPI interface, report persistence, verdict caching, sample source files, automated tests and setup scripts. **Verified so far:** 34 software-level tests, React build and selected browser/API interactions. **Remaining:** Ubuntu runtime acceptance, real model calibration, genuine sample output collection, team code review, repository publication and the assignment demonstration video.

Each owner should review another member's module using the shared contracts. A change to a metric must include its rule definition, evidence format and report behavior. The team should retain model/image identities and the verdict cache for the final demonstration.

8. Usefulness, limitations and discussion

8.1 Four practical applications

University programming laboratories. A class of 300 addition submissions can be checked against the same boundary tests. Students receive a specific failing input and expected sum instead of a generic wrong-answer message.

Self-directed programming practice. A learner submits a hanging program, receives a timeout indication, fixes termination and reruns the same rubric without waiting for an instructor.

Interview review assistance. Reviewers can compare per-metric evidence for 50 C exercises and concentrate human attention on reasoning, design tradeoffs and borderline decisions. The score should support, not replace, consequential human judgment.

Reference-copying triage. A high-similarity match against a locally saved tutorial, peer solution or known generated example identifies a concrete pair for manual inspection. This narrows review effort without asserting authorship.

8.2 Four concrete unreliable-verdict cases

Finite test coverage. A submission hard-codes the known and generated cases, earns a perfect test score and fails an unseen valid pair. Additional hidden/random tests reduce this risk, but passing finite tests does not prove correctness.

Model reasoning error. A loop hidden inside a macro may be incorrectly classified as constant time; a conventional mathematical variable name may be penalized unfairly. Instructor calibration and inspection of line-based findings remain necessary.

Authorship ambiguity. Two independently written five-line addition programs may exceed a similarity threshold, while renamed copied code can evade it. Unseen online copying and LLM authorship cannot be established from this local comparison alone.

Runtime and inference variability. A program near the timeout boundary may pass on an idle machine and fail under load. Sanitizer overhead may cause an instrumented-only failure. Temperature zero cannot guarantee identical fresh model judgments across versions or hardware; cache replay preserves a verdict, not its correctness.

8.3 Security boundary and responsible interpretation

The Docker design limits resources, disables networking and host mounts, uses a read-only root, and runs compiler/student children under an unprivileged identity. Nevertheless, containers share the host kernel. A public hostile-code judge would require stronger isolation, authentication, quotas and operational hardening. The current product is a local instructor prototype. Likewise, the language model is a rubric reviewer, not an authority that can override test evidence or prove absence of defects.

9. Deployment, conclusion and references

9.1 Reproducible Ubuntu deployment

The target environment requires Python 3.10+, Node.js 22.12+, Docker Engine and Ollama. Node builds the React assets; one Python process subsequently serves both the frontend and the API. Package, image and model downloads require internet during setup. Evaluation then uses the local model and sandbox.

```
# Run from the extracted autograder directory on Ubuntu.
bash scripts/setup.sh
ollama pull qwen2.5-coder:7b
docker build -t autograder-sandbox:1 sandbox/
.venv/bin/python app.py doctor
bash scripts/start.sh
# Open http://127.0.0.1:7860
```

For acceptance, run the full test suite with `RUN_DOCKER_TESTS=1 RUN_E2E_TESTS=1`, then execute `run_samples.py`. Archive real sample reports, rules, source, model/image identities and the verdict cache. Exact student scores should be reported only after these executions; none are invented in this paper.

9.2 Conclusion

AutoGrader combines modular software engineering with bounded local AI review to make C-program assessment configurable and inspectable. Its React interface, Python API, independent analyzers and JSON reports provide an end-to-end implementation structure aligned with the assignment. The completed software-level checks support the integration design, while Ubuntu sandbox validation and real-model calibration remain essential before claiming full grading performance. The next milestone is an evidence-backed demonstration using genuine sample evaluations and team review of every metric.

References

[1] *AutoGrader: A Rule-Driven Evaluator for C Programs.* Supplied Agentic-AI-assignment.pdf, pp. 1-9. Project requirements and illustrative rubric.

[2] TEAM EVEREST project artifacts. `docs/REPORT.md`, `docs/VALIDATION.md`, `rules/add.json` and implementation source. Local build/test record, 28 October 2026.

[3] Ollama. *Generate a chat message* and *Structured outputs.* docs.ollama.com/api/chat; docs.ollama.com/capabilities/structured-outputs.

[4] Docker. *Running containers* and *Docker Engine security.* docs.docker.com/engine/containers/run/; docs.docker.com/engine/security/.

[5] FastAPI documentation. fastapi.tiangolo.com. API implementation reference.

[6] React documentation and Vite guide. react.dev; vite.dev/guide/. Frontend and build references.

Document note: This paper describes the implemented project and documented progress. The proposed team allocation and schedule should be confirmed by TEAM EVEREST before institutional submission. No institutional affiliation, supervisor, repository URL or experimental grading outcome has been invented.